

Kruskal's MST

Minimum Spanning Trees, Kruskal's Algorithm · Mentor-Led Lab (Student Edition)

Developed by TalenciaGlobal

Difficulty ★★★ Advanced

Problem

Compute a minimum spanning tree with Kruskal's algorithm: report its total weight, the number of edges used, and whether the graph is connected. You are given starter code with two functions. `kruskal` is almost complete but has a bug, and `is_connected` is an empty stub. Fix the bug and implement the stub.

What to Compute

- **MST Weight:** the total weight of the minimum spanning tree (or forest).
- **Cables:** the number of edges chosen.
- **Connected:** Yes if the tree has $N-1$ edges, otherwise No.

Input / Output Format

Input: Line 1: N M . Next M lines: an undirected weighted edge u v w .

Output: Three lines: MST Weight, Cables, Connected.

Examples

Example 1

Input: 4 5 0 1 1 0 2 3 1 2 1 1 3 4 2 3 2

Output:

MST Weight: 4
Cables: 3
Connected: Yes

Explanation: Choosing 0-1 (1), 1-2 (1) and 2-3 (2) connects all four vertices for total weight 4 using 3 cables.

Example 2

Input: 3 3 0 1 1 1 2 2 0 2 3

Output:

MST Weight: 3
Cables: 2
Connected: Yes

Explanation: The two cheapest edges 0-1 (1) and 1-2 (2) span the triangle for weight 3.

Constraints

- Sort edges by (weight, u, v) for a deterministic result.
- Add an edge only if it joins two different groups.
- A connected graph uses exactly N-1 edges.

Your Task

- Task 1 - Fix the bug. `kruskal` is flagged with a comment, but the bug's location is not given. Find the single bug so an edge whose endpoints are already connected is SKIPPED (it would form a cycle).
- Task 2 - Implement `is_connected` from scratch: the graph is connected iff the spanning tree has exactly N-1 edges.

Starter Code

Begin from the code below. One function carries a single bug (flagged by a comment, with no hint to its location); the other is an empty stub you must implement.

Python

```
import sys

def find(parent, x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]
        x = parent[x]
    return x

# BUG: this function contains a single bug. Kruskal's algorithm must SKIP an edge whose
# endpoints are already connected (it would form a cycle); this version takes every edge.
def kruskal(n, edges):
    edges = sorted(edges, key=lambda e: (e[2], e[0], e[1]))
    parent = list(range(n))
    total = 0
    count = 0
    for u, v, w in edges:
        ru = find(parent, u)
        rv = find(parent, v)
        parent[ru] = rv
        total += w
        count += 1
    return total, count

# TODO: implement this. The graph is connected iff a spanning tree has exactly n-1 edges.
# Currently a stub.
def is_connected(n, count):
```

```

    return False

def main():
    data = [int(x) for x in sys.stdin.read().split()]
    n = data[0]; m = data[1]
    idx = 2
    edges = []
    for _ in range(m):
        edges.append((data[idx], data[idx + 1], data[idx + 2]))
        idx += 3
    total, count = kruskal(n, edges)
    print("MST Weight: " + str(total))
    print("Cables: " + str(count))
    print("Connected: " + ("Yes" if is_connected(n, count) else "No"))

main()

```

C++

```

// MST Lab 3 - Kruskal's MST (STARTER)
#include <iostream>
#include <vector>
#include <string>
#include <queue>
#include <algorithm>
using namespace std;

struct Edge {
    int u;
    int v;
    long long w;
};

bool edgeLess(const Edge& a, const Edge& b) {
    if (a.w != b.w) {
        return a.w < b.w;
    }
    if (a.u != b.u) {
        return a.u < b.u;
    }
    return a.v < b.v;
}

int findRoot(vector<int>& parent, int x) {
    while (parent[x] != x) {
        parent[x] = parent[parent[x]];
        x = parent[x];
    }
    return x;
}

// BUG: this function contains a single bug. Kruskal's algorithm must SKIP an edge whose
// endpoints are already connected (it would form a cycle); this version takes every edge.
void kruskal(int n, vector<Edge>& edges, long long& total, int& count) {
    sort(edges.begin(), edges.end(), edgeLess);
    vector<int> parent(n);
    for (int i = 0; i < n; i++) {

```

```

    parent[i] = i;
}
total = 0;
count = 0;
for (const Edge& e : edges) {
    int ru = findRoot(parent, e.u);
    int rv = findRoot(parent, e.v);
    parent[ru] = rv;
    total += e.w;
    count++;
}
}

// TODO: implement this. The graph is connected iff a spanning tree has exactly n-1 edges.
// Currently a stub.
bool isConnected(int n, int count) {
    return false;
}

int main() {
    int n, m;
    cin >> n >> m;
    vector<Edge> edges(m);
    for (int i = 0; i < m; i++) {
        cin >> edges[i].u >> edges[i].v >> edges[i].w;
    }
    long long total;
    int count;
    kruskal(n, edges, total, count);
    cout << "MST Weight: " << total << "\n";
    cout << "Cables: " << count << "\n";
    cout << "Connected: " << (isConnected(n, count) ? "Yes" : "No") << "\n";
    return 0;
}

```

Java

```

// MST Lab 3 - Kruskal's MST (STARTER)
import java.util.*;

public class Main {
    static class Edge {
        int u, v;
        long w;
        Edge(int u, int v, long w) {
            this.u = u;
            this.v = v;
            this.w = w;
        }
    }

    static Comparator<Edge> edgeOrder = (a, b) -> {
        if (a.w != b.w) {
            return Long.compare(a.w, b.w);
        }
        if (a.u != b.u) {
            return Integer.compare(a.u, b.u);
        }
    }
}

```

```

    }
    return Integer.compare(a.v, b.v);
};

static int find(int[] parent, int x) {
    while (parent[x] != x) {
        parent[x] = parent[parent[x]];
        x = parent[x];
    }
    return x;
}

// BUG: this function contains a single bug. Kruskal's algorithm must SKIP an edge whose
// endpoints are already connected (it would form a cycle); this version takes every edge.
static long[] kruskal(int n, List<Edge> edges) {
    edges.sort(edgeOrder);
    int[] parent = new int[n];
    for (int i = 0; i < n; i++) {
        parent[i] = i;
    }
    long total = 0;
    int count = 0;
    for (Edge e : edges) {
        int ru = find(parent, e.u);
        int rv = find(parent, e.v);
        parent[ru] = rv;
        total += e.w;
        count++;
    }
    return new long[] { total, count };
}

// TODO: implement this. The graph is connected iff a spanning tree has exactly n-1 edges.
// Currently a stub.
static boolean isConnected(int n, long count) {
    return false;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int m = sc.nextInt();
    List<Edge> edges = new ArrayList<>();
    for (int i = 0; i < m; i++) {
        edges.add(new Edge(sc.nextInt(), sc.nextInt(), sc.nextLong()));
    }
    long[] r = kruskal(n, edges);
    System.out.println("MST Weight: " + r[0]);
    System.out.println("Cables: " + r[1]);
    System.out.println("Connected: " + (isConnected(n, r[1]) ? "Yes" : "No"));
}
}

```

Sample Run

Sample Input

```
4 5
0 1 1
0 2 3
1 2 1
1 3 4
2 3 2
```

Sample Output

```
MST Weight: 4
Cables: 3
Connected: Yes
```